

Vision-Based Desktop Automation with Dynamic Icon Grounding

Lorelei Lee Lopez

Objective:

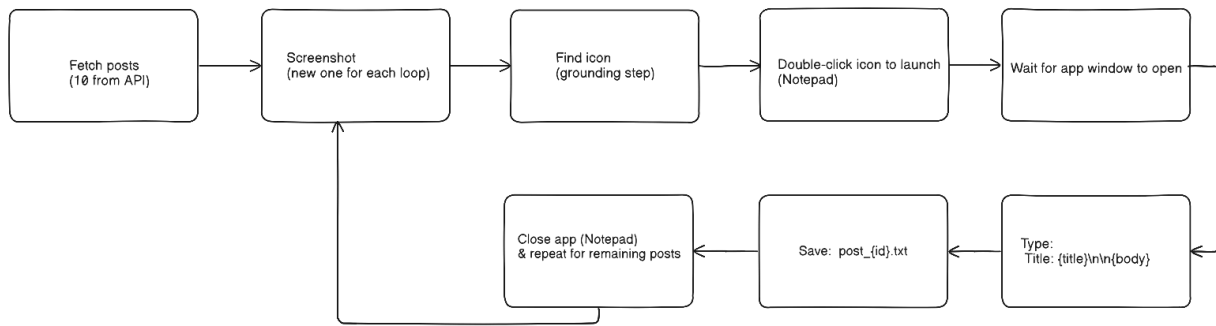
The goal of this project is to create a Python application that can locate and open the Notepad icon on a Windows desktop no matter its location on the screen, using computer vision. This will be done by taking a screenshot and searching within the screenshot using a text description of said icon. This program will not rely on a saved image for reference. For this particular assignment, the Notepad icon will be the targeted icon, however it can be used for locating and opening other icons as well. This was tested and proven to be able to locate the Recycle Bin icon.

Assumptions:

- Windows 11 using a resolution of 1920x1080 with 100% scaling, on a single monitor.
- A Notepad shortcut exists on the desktop and is visible, specifically the Windows 11 version with tabs.
- Internet Access to connect with: JSONPlaceholder API and Claude API with a valid API key.
- Icon size and position will vary between runs and was tested in different positions, however size was not tested.
- Desktop themes can vary between light and dark modes This was tested once to show Dark mode capabilities.
- Unforeseen pop-ups may appear.
- The screen must remain on and unlocked while the program is running.

System design:

When the program begins, first it will fetch the first 10 posts from the API. Next, for each post that was fetched, it shows the desktop, moves the mouse out of the way, takes a screenshot and searches for the Notepad icon using a text description of the icon. Once the icon is located it will double-click the icon and wait for it to open. Once it is open and it confirms the tab is blank, it will type in the blank note "Title: {title}\n\n{body}" in the text area. Next, it will save it as post_{id}.txt in the tjm-project folder, and close the Notepad. This is repeated from a fresh screenshot for the next post.



Grounding strategy:

To better understand why a grounding strategy is necessary to locate an icon on the screen, first it is necessary to understand where a vision model lacks. A vision model will run into difficulty locating an icon due to a few reasons:

- Screen vs icon size: Icons are often very small in comparison to the desktop size they sit in, as that gap widens, accuracy falls.
- Screenshot limitations: Depending on the size of the screenshot it may be required to be scaled down in order to be appropriately ingested by the model. This shrinking will further decrease icon size and cause finer details to be lost.
- Clutter: Icons, such as Notepad, do not exist alone, desktops can be filled and become cluttered with various icons including similar looking ones.

Based on “ScreenSpot-Pro: GUI Grounding for Professional High-Resolution

Computer Use” by Li et al, a grounding strategy can help mitigate these drawbacks and increase accuracy of locating the intended icon. This can be accomplished using similar steps with a few changes for the scope of this project:

- Handling pop-ups: Before beginning its search the planner will evaluate if a pop-up is blocking the desktop. If there is, only a control from a fixed list of close or dismiss labels can be clicked, anything outside of that will stop the program from running. This was only tested with simulated pop-ups.
- Inference: The planner model, in this case Claude, would look at the screenshot and the description of the icon and list as text, where the icon is the most likely to be and what sits near it. Oftentimes, neighboring elements can be easier to spot than the intended icon target to help point the model in the right direction.
- Simplified scoring: Each of Claude's guesses is located on the screenshot and expanded into a search region. Regions that several guesses agree on rank highest, as in the paper. Overlapping regions are merged, and the program searches them from best to worst.
- Recursive search: The highest scoring candidate is cropped and searched again by zooming in so the icon takes up more of the image until the intended region is 300 px or smaller and the icon is pointed at directly. If the icon is not found, it moves onto the next region. It keeps repeating this process until either the icon is found, or a depth of 3 or 40 model calls are reached.

- Coordinates: When the intended icon is located the grounder points to where it is on the cropped screenshot and then returns the coordinates on the full screenshot to show location on the screen. When the description says "not the taskbar," anything inside the taskbar area is rejected
- Verify: The verifier looking at a close-up of the point with a box drawn on it, decides if this is the target, if the target is somewhere else, or if it is not found. If it is not found it will move on to the next highest scoring candidate. When the icon is found it will return the (x,y) coordinates needed to locate the icon. If every candidate fails, the program stops, saves a debug screenshot, and reports an error instead of guessing

For this particular project Claude will be used as the planner, grounder and verifier. Each will have separate prompts and Sonnet 5.5 with medium effort will be used. This is a different approach to the paper referenced above which used GPT-4o as the planner and a separate model trained for pointing as the grounder. Because Claude is not a dedicated pointing model the search zooms into a small region, and the verifier confirms the icon before the program clicks. I chose this setup because of my constraints: the time limit, the tools readily available, token use, and the limits of my laptop.

Alternatives and Tradeoffs:

Other simpler methods could be used to search for an icon. However while they excel in simplicity they lack flexibility.

- Matching the exact icon image: Icons can vary in size, theme, and more. This would require a pre-saved copy of the icon image ahead of time thus it would not allow for the program to be used across different desktops or if the desktop theme changed.
- OCR: If the label is not available or is using a different name, it would be unable to locate the icon across different desktops.

The approach chosen for this project is intended to work across various desktops, unlike the above two alternatives, it will be slower, cost money, and can fail due to the model making mistakes. This is why the verification step exists to help decrease the chances of that happening. However, those drawbacks can be improved upon in later iteration with more time and resources while still maintaining use across desktops.

Error Handling:

Similar to my approach when planning with Claude Code, I ensured that the program is checking before acting and if it runs into an error it stops.

Failures encountered along the way:

- Icon found in taskbar: A text filter dropped the right region when the icon was near the taskbar. I replaced it with a check on the taskbar area itself.

- Region blindness: The search only tried the top 3 regions even though the icon was in the 4th. Now it tries every region.
- Save As dialog: The dialog stayed open with the path typed. The program now reads the File name box back and retries with Alt+S.
- Spell-check: Notepad's autocorrect changed "commodi" to "commode," which caused the program to stop. Now each word is checked and repaired in place.
- Wrong icon: The grounder once guessed the wrong icon when it couldn't find Notepad, and the verifier rejected it. Rejected candidates are skipped and the search moves on.

Failures that were anticipated but not encountered:

- Coordinate off screen: If the program detects that the coordinates suggested are outside of the screen or cropped screenshot it will reject it.
- Connection issues: If the program can't reach JSONPlaceholder, it retries up to 3 times, then stops with an error. If it can't reach the Claude API, the SDK retries twice, then that step counts as "not found," so the search ends with an error and nothing is clicked.

Future Improvements:

For a future iteration the program could remember the last location and use that as a starting point for the next time. This would be used as a hint to try and improve speed. Each loop would still use a fresh screenshot and verify the result, this would just be to try and cut down on the amount of search steps needed.